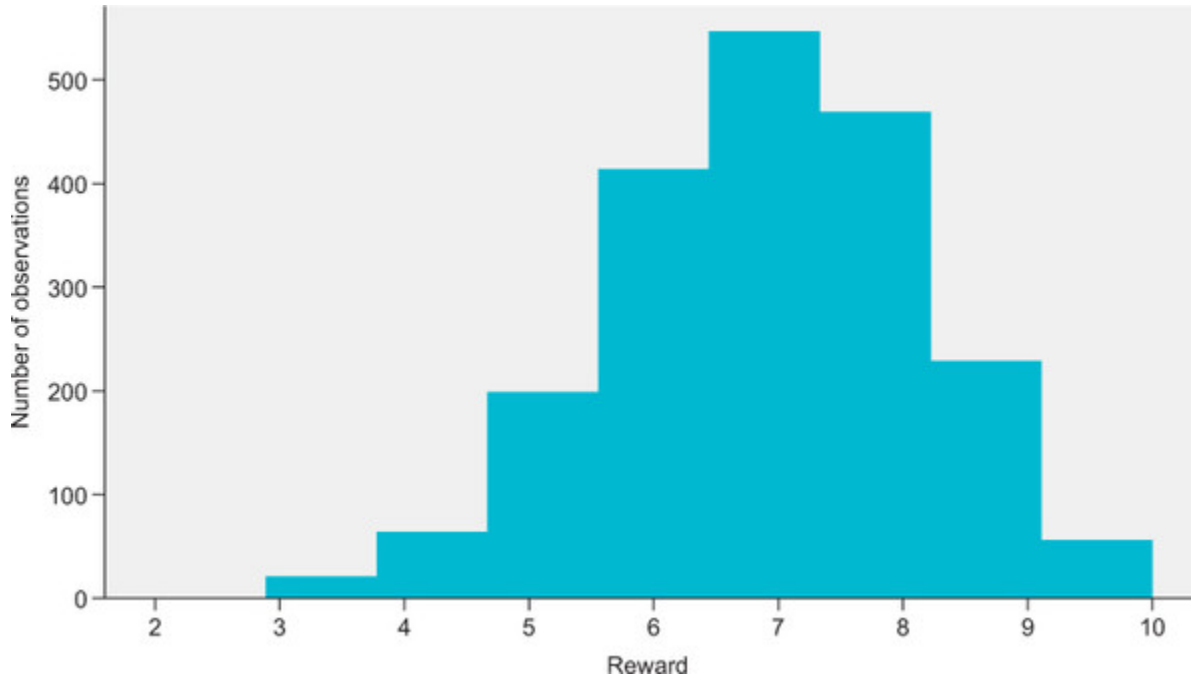
```
def get_reward(prob, n=10):
    reward = 0
    for i in range(n):
        if random.random() < prob:
            reward += 1
    return reward
```

You can check this by running it:

```
>>> np.mean([get_reward(0.7) for _ in range(2000)])
7.001
```

This output shows that running this code 2,000 times with a probability of 0.7 indeed gives us a mean reward of close to 7 (see the histogram in figure 2.2).

**Figure 2.2. The distribution of rewards for a simulated n-armed bandit with a 0.7 probability of payout.**




The next function we'll define is our greedy strategy of choosing the best arm so far. We need a way to keep track of which arms were pulled and what the resulting reward was. Naively, we could just have a list and append observations such as (arm, reward), e.g., (2, 9), indicating we chose